Aragon - Aragon OSx 1.4

Aragon

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Dec 1, 2024 - Jan 3, 2025

SOURCE CODE
- osx
- multisig-plugin
- admin-plugin
- token-voting-plugin
- staged-proposal-processor-plugin
- osx-commons

ASSESSED COMMIT ID
- e0ba7b6
- fffc680
- 546cfa2
- 02a7dbb
- dca24e2
- 4eca1de

SEVERITY BREAKDOWN

All Findings	Solved	Risk Accepted	Acknowledged
7	3	1	3



Introduction

Aragon engaged Halborn to conduct a security assessment on their smart contracts beginning on December 2nd, 2024 and ending on January 3rd, 2025. The security assessment was scoped to the smart contracts provided to the Halborn team.
Commit hashes and further details can be found in the Scope section of this report.

The Aragon DAO is a set of programmable decentralised autonomous organisation contracts and plugins enabling different types of consensus needed to pass proposals (token voting, multisig...).

The current assessment covers a big part of the scope that was already reviewed previously, but also focuses on minor contract upgrades and the addition of a new plugin, the staged proposal plugin.

Assessment Summary

The team at `Halborn` assigned a full-time security engineer to assess the security of the smart contracts. The security engineer is a blockchain and smart-contract security expert with advanced penetration testing, smart-contract hacking, and deep knowledge of multiple blockchain protocols. The purpose of this assessment is to:

- Ensure that smart contract functions operate as intended.
- Identify potential security issues with the smart contracts.
- Ensure upgrades are not breaking the contracts.
- Provide recommendations on how to handle loss of availability in sub bodies of the staged proposal processor.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were partially addressed by the `Aragon team`. The main ones were the following:

- `Make staged proposal plugin robust against sub bodies denial of availability.`
- `Introduce a buffer between end of voting and limit time for the proposal execution.`
- `Introduce incentives for token voters to hold the tokens after a vote.`
- `Fix unreachable conditions.`

Test Approach and Methodology

`Halborn` performed a combination of manual review of the code and automated security testing to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of the smart contract assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance coverage of smart contracts and can quickly identify items that do not follow security best practices. The following phases and associated tools were used throughout the term of the assessment:

- Research into the architecture, purpose, and use of the platform.
- Smart contract manual code review and walkthrough to identify any logic issue.

- Thorough assessment of safety and usage of critical Solidity variables and functions in scope that could lead to arithmetic related vulnerabilities.
- Manual testing by custom scripts.
- Graphing out functionality and contract logic/connectivity/functions (`solgraph`).
- Static Analysis of security for scoped contract, and imported functions. (`Slither`).
- Local or public testnet deployment (`Foundry`, `Remix IDE`)

Caveats

After the initial assessment, `Aragon` reported an additional potential bypass using flash loans, affecting the `minProposerVotingPower` check.

Halborn performed a post-assessment review of PR14 (commit `c30c2de`), and confirms the remediation is present in that commit.

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

5.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

5.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

5.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

REPOSITORIES

(a) Repository: osx

(b) Assessed Commit ID: e0ba7b60b08fa1665ecac92dc12ea89e4245e7dc

(c) Items in scope:

- osx/packages/contracts/src 17 files
  - core 4 files
    - dao 2 files
      - DAO.sol Solidity
      - IEIP4824.sol Solidity
    - permission 1 file
      - PermissionManager.sol Solidity
    - utils 1 file
      - CallbackHandler.sol Solidity
  - framework 13 files
    - dao 2 files
      - DAOFactory.sol Solidity
      - DAORegistry.sol Solidity
    - plugin 7 files
      - repo 5 files
        - placeholder 1 file
          - PlaceholderSetup.sol Solidity
        - IPluginRepo.sol Solidity
        - PluginRepo.sol Solidity
        - PluginRepoFactory.sol Solidity
        - PluginRepoRegistry.sol Solidity
      - setup 2 files
        - PluginSetupProcessor.sol Solidity
        - PluginSetupProcessorHelpers.sol Solidity
    - utils 4 files
      - ens 2 files
        - ENSMigration.sol Solidity
        - ENSSubdomainRegistrar.sol Solidity
      - InterfaceBasedRegistry.sol Solidity
      - RegistryUtils.sol Solidity

Additional scope: fc7403fcef660aa952c7e635e95257ee7f466efb

Out-of-Scope: Third party dependencies. , Economic attacks.

(a) Repository: multisig-plugin

(b) Assessed Commit ID: fffc680f563698cfb7aec962fb89b4196025f629

(c) Items in scope:

- multisig-plugin/packages/contracts/src 4 files
  - IMultisig.sol Solidity
  - ListedCheckCondition.sol Solidity
  - Multisig.sol Solidity
  - MultisigSetup.sol Solidity

Additional scope: 69874ebe11efd0e9687ba24a7e04ba7fac08012d

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: admin-plugin

(b) Assessed Commit ID: 546cfa243a5d0726d75158db646573ca2237f570

(c) Items in scope:

- admin-plugin/packages/contracts/src 2 files
  - Admin.sol Solidity
  - AdminSetup.sol Solidity

Additional scope: 81db7a26668551a91077fcc23efc10dd937fcfe7

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: token-voting-plugin

(b) Assessed Commit ID: 02a7dbb95c42ebd2226117bf85a0fe330c788948

(c) Items in scope:

- token-voting-plugin/packages/contracts/src 9 files
  - ERC20 4 files
    - governance 3 files
      - GovernanceERC20.sol Solidity
      - GgovernanceWrappedERC20.sol Solidity
      - IGovernanceWrappedERC20.sol Solidity
    - IERC20MintableUpgradeable.sol Solidity
  - IMajorityVoting.sol Solidity
  - MajorityVotingBase.sol Solidity
  - TokenVoting.sol Solidity
  - TokenVotingSetup.sol Solidity
  - VotingPowerCondition.sol Solidity

Additional scope: 9eb9b15986db2fcb9a02c89e3aa99fe8c0292a1c

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: staged-proposal-processor-plugin

(b) Assessed Commit ID: dca24e2db5625d9898c29c9d579873442879dcf3

(c) Items in scope:

- staged-proposal-processor-plugin/src 6 files

- libraries 1 file
  - Errors.sol Solidity
- utils 3 files
  - PluginSettings.sol Solidity
  - SPPRuleCondition.sol Solidity
  - TrustedForwarder.sol Solidity
- StagedProposalProcessor.sol Solidity
- StagedProposalProcessorSetup.sol Solidity

Additional scope: ad73f165e5d7204a37c404d2bd270e401a127dc7

Out-of-Scope: Third party dependencies and economic attacks.

(a) Repository: osx-commons

(b) Assessed Commit ID: 4eca1de80e60f5f516e8f15b45f91e05ea5348e8

(c) Items in scope:

- osx-commons/contracts/src 33 files
  - dao 1 file
    - IDAO.sol Solidity
  - executors 2 files
    - Executor.sol Solidity
    - IExecutor.sol Solidity
  - permission 8 files
    - auth 3 files
      - auth.sol Solidity
      - DaoAuthorizable.sol Solidity
      - DaoAuthorizableUpgradeable.sol Solidity
    - condition 4 files
      - extensions 1 file
        - RuledCondition.sol Solidity
      - IPermissionCondition.sol Solidity
      - PermissionCondition.sol Solidity
      - PermissionConditionUpgradeable.sol Solidity
    - PermissionLib.sol Solidity
  - plugin 12 files
    - extensions 5 files
      - governance 1 file
        - Addresslist.sol Solidity
      - membership 1 file
        - IMembership.sol Solidity
      - proposal 3 files
        - IProposal.sol Solidity
        - Proposal.sol Solidity
        - ProposalUpgradeable.sol Solidity
    - setup 3 files
      - IPluginSetup.sol Solidity
      - PluginSetup.sol Solidity
      - PluginUpgradeableSetup.sol Solidity
    - IPlugin.sol Solidity
    - Plugin.sol Solidity

- PluginCloneable.sol Solidity
- PluginUUPSUpgradeable.sol Solidity
- utils 10 files
  - deployment 2 files
    - ProxyFactory.sol Solidity
    - ProxyLib.sol Solidity
  - math 3 files
    - BitMap.sol Solidity
    - Ratio.sol Solidity
    - UncheckedMath.sol Solidity
  - metadata 2 files
    - MetadataExtension.sol Solidity
    - MetadataExtensionUpgradeable.sol Solidity
  - versioning 3 files
    - IProtocolVersion.sol Solidity
    - ProtocolVersion.sol Solidity
    - VersionComparisonLib.sol Solidity

Additional scope: 869045b069679fad94d640a8a3e1a1b5538cf0f0

REMEDIATION COMMIT ID:

590ebd8d5931cb6811ea55ca9e6e3f96df6969b3

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Proposal creation threshold can be bypassed via flash loans	High	8.4	SOLVED 08/20/2025
HAL-002	StagedProposalProcessor plugin can lose ability to advance	Medium	6.3	RISK ACCEPTED 01/14/2025
HAL-003	Suceeded proposals may expire before execution	Informational	1.7	ACKNOWLEDGED 01/20/2025
HAL-004	Permission inconsistency between StagedProposalProcessorSetupZkSync and	Informational	1.7	SOLVED 01/30/2025

#	TITLE	SEVERITY	SCORE	STATUS
	StagedProposalProcessorSetup			
HAL-005	No incentive to hold voting tokens after a vote	Informational	—	ACKNOWLEDGED 01/14/2025
HAL-006	Unreachable condition in StagedProposalProcessor	Informational	—	SOLVED 11/22/2024
HAL-007	Anyone can report a proposal result	Informational	—	ACKNOWLEDGED 01/14/2025

Findings & Tech Details

HAL-001 SOLVED

Proposal creation threshold can be bypassed via flash loans

High · 8.4 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:N/I:C/D:M/Y:M

DESCRIPTION

`Aragon team` reported a post-assessment finding, indicating that the `VotingPowerCondition.isGranted()` function in `VotingPowerCondition.sol` allowed the `minProposerVotingPower` check to be bypassed using flash loans or instant delegation. The original implementation checked the current voting power (`getVotes()`) or token balance (`balanceOf()`), both of which can be temporarily inflated within a single block.

SOL

```
if (minProposerVotingPower_ != 0) {
    if (
        VOTING_TOKEN.getVotes(_who) < minProposerVotingPower_
            && IERC20Upgradeable(address(VOTING_TOKEN)).balanceOf(_who) <
minProposerVotingPower_
    ) {
        return false;
    }
}
```

As a result, any account could use a flash loan to momentarily meet the proposal threshold and create a proposal, even if they do not hold sufficient voting power at the relevant snapshot.

For example:

1. An attacker obtains a flash loan to temporarily increase their token balance.
2. The attacker calls the proposal creation function, passing the `minProposerVotingPower` check.
3. The flash loan is repaid in the same block, but the proposal remains valid.

RECOMMENDATION

Validate proposer eligibility using `VOTING_TOKEN.getPastVotes(_who, timepoint)`, where `timepoint` is set to the previous block or timestamp, depending on token indexing. This ensures the check is based on immutable, historical voting power and cannot be manipulated within the same block.

REMEDIATION COMMENT

Addressed in: 590ebd8d5931cb6811ea55ca9e6e3f96df6969b3

SOLVED: The `Aragon team` solved this finding in the specified commit by following the mentioned recommendation.

SOL

```sol
if (minProposerVotingPower_ != 0) {
    uint256 _timepoint;
    if (TOKEN_VOTING.tokenIndexedByTimestamp()) {
        _timepoint = block.timestamp - 1;
    } else {
        _timepoint = block.number - 1;
    }

    if (VOTING_TOKEN.getPastVotes(_who, _timepoint) < minProposerVotingPower_) {
        return false;
    }
}
```

HAL-002 RISK ACCEPTED

StagedProposalProcessor plugin can lose ability to advance

Medium · 6.3 AO:A/AC:L/AX:M/R:N/S:C/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

The **StagedProposalProcessor plugin (SPP)** organizes a proposal lifecycle around different stages, of multiple bodies. For example, stage 1 can be **TokenVotingPlugin** + **MultisigPlugin**, and stage 2 can be **AdminPlugin**. Each stage has a fixed length and waits for the result of the majority of the bodies to advance to the next stage.

Naturally, when installing this plugin, the other installed plugins must be revoked of their rights, since it does not make sense to have a short path (like a Multisig) and a long path (SPP composed of multisig, token voting...) for the same power.

Identified Issues

A few issues can arise from the SPP configuration, as caught by the Aragon team prior to the assessment:

1. **Permanent Unresponsive Body**
   - A body can be permanently unresponding: for example, a multisig where users lost access to their keys, a huge token holder could lose access to their keys, or simply not participate in governance anymore.
2. **Configuration Conflicts**
   - A body configuration or update can conflict with the SPP: for example, the fixed length of the stages in the SPP can differ from the minimum or maximum length of the sub-bodies.
3. **Proposal Creation Failures**
   - If a sub-body's proposal creation is extremely expensive in gas (or has an unexpected revert path), the SPP proposal creation function might fail due to out-of-gas or reverts.

Proposed Solutions

The proposed solution from the Aragon team is to introduce a **recover functionality** allowing the removal of a bricked body and adjust thresholds (either the number of bodies needed for approval or the number of vetoes allowed). This solves the possible loss of availability at the cost of a

possible takeover from a plugin with enough rights. For instance, a malicious majority could remove a vetoing sub-body from the next stage just because they “don’t like” the potential veto. Similarly, they could remove sub-bodies that are meant to be necessary checks/balances.

`Recover Function Design`
Having a recover function only callable by an administrator or a multisig, after a certain condition (enough time since the last succeeded proposal, number of failed proposals...), seems to be a simple and robust way of handling the recovery, although still carrying centralization risks (e.g., malicious administrators or loss of keys).

`Configuration Validation`
One might still argue that the best approach is simply to keep things simple. If the operator or DAO incorrectly sets the stage vote duration to 20 minutes, while the sub-body has a minimum duration of 1 hour, that is a config mistake which should be caught in an update stage validation step before the plugin is used in production.

`Preventing Harmful Upgrades`
It might be interesting to prevent harmful upgrades in sub-bodies. For example, any upgrade on the vote duration that would not match the SPP config should be refused. This could be implemented with a callback from the sub-body to an `upgradeValidator` address (being the SPP contract), checking the validity of the update parameters.

`Handling Proposal Creation Failures`
In case of failure to create a proposal in a sub-body, a partial success loop could be used, where the SPP tries to create proposals for each body in a loop, keeping track of the successful indices and skipping the failures. The same reasoning could be employed where the plugins expose an `isBricked` method that serves the SPP to exclude the plugin from the approvers or creating proposals on it. Note that this kind of failure should be detected priorly during tests so it should be rare in production.

`Safe Partial Creation and Non-Upgradeable Sub-Bodies`
Another "middle ground" idea is to add “safe partial creation” or to require that sub-bodies are themselves non-upgradeable once they have been integrated into SPP. That way, parameters can’t get out-of-sync mid-flight.

Improved Recover Function Design

The current recover function proposed in the recover-proposal tag does only prevent issues caused by the voting duration, and it might be a better design to prevent durations mismatches with additional verifications during setups or upgrades, rather than a post-issue fix.

Conclusion

Addressing the potential issues with the StagedProposalProcessor involves a combination of introducing robust recovery mechanisms, validating configurations upfront, preventing harmful upgrades, and ensuring proposal creation reliability. While these solutions enhance the resilience and integrity of the governance process, they also introduce certain centralization risks that must be carefully managed.

Code Location

The following snippets show an example where the `TokenVotingPlugin` could refuse the proposal creations.

Enforcing Minimum/Maximum Durations

This snippet shows that durations can be updated by a plugin, independently from the SPP durations.

- token-voting-plugin/packages/contracts/src/MajorityVotingBase.sol:

SOL 700–720

```
700 function _updateVotingSettings(VotingSettings calldata _votingSettings) internal
    virtual {
701     // ...
702
703     if (_votingSettings.minDuration < 60 minutes) {
704         revert MinDurationOutOfBounds({limit: 60 minutes, actual:
    _votingSettings.minDuration});
705     }
706
707     if (_votingSettings.minDuration > 365 days) {
708         revert MinDurationOutOfBounds({limit: 365 days, actual:
    _votingSettings.minDuration});
709     }
710
711     votingSettings = _votingSettings;
712
713     emit VotingSettingsUpdated({
714         votingMode: _votingSettings.votingMode,
715         supportThreshold: _votingSettings.supportThreshold,
716         minParticipation: _votingSettings.minParticipation,
717         minDuration: _votingSettings.minDuration,
718         minProposerVotingPower: _votingSettings.minProposerVotingPower
    });
719 }
720
```

Validating Proposal Start and End Dates

These lines show that dates proposed by the SPP could be considered out of bounds if the durations mismatch.

- token-voting-plugin/packages/contracts/src/MajorityVotingBase.sol:

SOL 761-788

```sol
function _validateProposalDates(
    uint64 _start,
    uint64 _end
) internal view virtual returns (uint64 startDate, uint64 endDate) {
    uint64 currentTimestamp = block.timestamp.toUint64();

    if (_start == 0) {
        startDate = currentTimestamp;
    } else {
        startDate = _start;

        if (startDate < currentTimestamp) {
            revert DateOutOfBounds({limit: currentTimestamp, actual: startDate});
        }
    }

    uint64 earliestEndDate = startDate + votingSettings.minDuration;

    if (_end == 0) {
        endDate = earliestEndDate;
    } else {
        endDate = _end;

        if (endDate < earliestEndDate) {
            revert DateOutOfBounds({limit: earliestEndDate, actual: endDate});
        }
    }
}
```

RECOMMENDATION

It is recommended to:

- Verify beforehand the compatibility of the voting timelines between the SPP and the sub bodies.
- Give the possibility to bodies to signal their state of availability, so the SPP can ignore them when in a long enough non-available state.

REFERENCES

[]

REMEDIATION COMMENT

RISK ACCEPTED: The **Aragon team** accepted the risk behind this finding. The remediation would introduce too much complexity and potential bugs, while the denial of service can be avoided by a thorough verification of the plugin settings before installing the **StagedProposalProcessor**.

HAL-003 ACKNOWLEDGED

Suceeded proposals may expire before execution

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

In the token majority voting plugin, a scenario could exist where the majority is reached at the last block before the end of the proposal (`proposal_.parameters.endDate`) and a manual execution of the proposal (not using `_tryEarlyExecution`) would cause the proposal to succeed but switch in a closed state.

Code Location

The following snippets show that the `vote` function checks the `_canVote` function, that checks the `_isProposalOpen` function, to highlight that the end of the vote matches the end limit for the execution.

- token-voting-plugin/packages/contracts/src/MajorityVotingBase.sol:

SOL 364-379

```
364 function vote(
365     uint256 _proposalId,
366     VoteOption _voteOption,
367     bool _tryEarlyExecution
368 ) public virtual {
369     address account = _msgSender();
370
371     if (!_canVote(_proposalId, account, _voteOption)) {
372         revert VoteCastForbidden({
373             proposalId: _proposalId,
374             account: account,
375             voteOption: _voteOption
376         });
377     }
378     _vote(_proposalId, _voteOption, account, _tryEarlyExecution);
    }
379
```

- token-voting-plugin/packages/contracts/src/TokenVoting.sol:

SOL 290-321

```
290 function _canVote(
291     uint256 _proposalId,
292     address _account,
293     VoteOption _voteOption
294 ) internal view override returns (bool) {
295     Proposal storage proposal_ = proposals[_proposalId];
296
297     // The proposal vote hasn't started or has already ended.
298     if (!_isProposalOpen(proposal_)) {
299
```

```
300             return false;
301         }
302
303         // The voter votes `None` which is not allowed.
304         if (_voteOption == VoteOption.None) {
305             return false;
306         }
307
308         // The voter has no voting power.
            if (votingToken.getPastVotes(_account, proposal_.parameters.snapshotBlock) == 0) {
309             return false;
310         }
311
312         // The voter has already voted but vote replacment is not allowed.
            if (
313             proposal_.voters[_account] != VoteOption.None &&
314             proposal_.parameters.votingMode != VotingMode.VoteReplacement
315         ) {
                return false;
316         }
317
318         return true;
319     }
320
```

- token-voting-plugin/packages/contracts/src/MajorityVotingBase.sol:

SOL 698-705

```
698 function _isProposalOpen(Proposal storage proposal_) internal view virtual returns
    (bool) {
699     uint64 currentTime = block.timestamp.toUint64();
700
701     return
702         proposal_.parameters.startDate <= currentTime &&
703         currentTime < proposal_.parameters.endDate &&
704         !proposal_.executed;
705 }
```

Same for the Multisig plugin:

- multisig-plugin/packages/contracts/src/Multisig.sol:

SOL 611-617

```
611 function _isProposalOpen(Proposal storage proposal_) internal view returns (bool) {
        uint64 currentTimestamp64 = block.timestamp.toUint64();
612     return
613         !proposal_.executed &&
614         proposal_.parameters.startDate <= currentTimestamp64 &&
615         proposal_.parameters.endDate >= currentTimestamp64;
616 }
617
```

RECOMMENDATION

It is recommended to add a buffer period between the end of votes and the execution limit.

REFERENCES

[]

REMEDIATION COMMENT

ACKNOWLEDGED: The **Aragon team** acknowledged this finding, also mentioning that:

> Introducing a buffer period could result in a scenario where voting is restricted during the buffer, even though the proposal has not yet been executed. Additionally, implementing such a buffer would introduce an unnecessary delay in the governance process, which we aim to avoid.
> Our current design choice ensures no hard delays while achieving the same overall outcome.

HAL-004 SOLVED

Permission inconsistency between StagedProposalProcessorSetupZkSync and StagedProposalProcessorSetup

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The **StagedProposalProcessorSetupZkSync.sol** contract is copied from **StagedProposalProcessorSetup.sol**, aimed for zkSync deployment. It was found that the `CANCEL_PERMISSION_ID` permission was granted by the zkSync setup contract, but not the original.

This creates inconsistencies between the StagedProposalProcessors deployed on both environments.

Code Location

- StagedProposalProcessorSetupZkSync.sol:

SOL 167-173

```
167 permissions[6] = PermissionLib.MultiTargetPermission({
168     operation: _op,
169     where: _spp,
170     who: ANY_ADDR,
171     condition: PermissionLib.NO_CONDITION,
172     permissionId: Permissions.CANCEL_PERMISSION_ID
173 });
```

RECOMMENDATION

It is recommended to review both files and keep the wanted version of the permissions.

REFERENCES

[]

REMEDIATION COMMENT

SOLVED: The issue was fixed in the specified commit.

HAL-005 ACKNOWLEDGED

No incentive to hold voting tokens after a vote

• Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

The voting plugin relies on a snapshot of the balance of the voting token holders at the block prior to the proposal creation to avoid manipulations such as flashloans. This, however, allows users to get rid of the voting tokens and still participate in the proposals with the recorded voting power at the proposal creation.

Code Location

- token-voting-plugin/packages/contracts/src/TokenVoting.sol:

SOL 238-287

```
238 function _vote(
239     uint256 _proposalId,
240     VoteOption _voteOption,
241     address _voter,
242     bool _tryEarlyExecution
243 ) internal override {
244     Proposal storage proposal_ = proposals[_proposalId];
245
246     // This could re-enter, though we can assume the governance token is not malicious
        uint256 votingPower = votingToken.getPastVotes(_voter,
247 proposal_.parameters.snapshotBlock);
        VoteOption state = proposal_.voters[_voter];
248
249     // If voter had previously voted, decrease count
250     if (state == VoteOption.Yes) {
251         proposal_.tally.yes = proposal_.tally.yes - votingPower;
252     } else if (state == VoteOption.No) {
253         proposal_.tally.no = proposal_.tally.no - votingPower;
254     } else if (state == VoteOption.Abstain) {
255         proposal_.tally.abstain = proposal_.tally.abstain - votingPower;
256     }

257     // write the updated/new vote for the voter.
258     if (_voteOption == VoteOption.Yes) {
259         proposal_.tally.yes = proposal_.tally.yes + votingPower;
260     } else if (_voteOption == VoteOption.No) {
261         proposal_.tally.no = proposal_.tally.no + votingPower;
262     } else if (_voteOption == VoteOption.Abstain) {
263         proposal_.tally.abstain = proposal_.tally.abstain + votingPower;
264     }
265
        proposal_.voters[_voter] = _voteOption;
266
267     emit VoteCast({
268         proposalId: _proposalId,
269         voter: _voter,
270         voteOption: _voteOption,
271         votingPower: votingPower
272     });
273
274     if (!_tryEarlyExecution) {
275         return;
276     }
277
```

```
278
279         if (
280             _canExecute(_proposalId) &&
281             dao().hasPermission(address(this), _voter, EXECUTE_PROPOSAL_PERMISSION_ID,
282     _msgData())
283         ) {
284             _execute(_proposalId);
285         }
286     }
```

RECOMMENDATION

It is recommended to use a locking mechanism to ensure that the tokens remain frozen until the proposal passes or fails.

REFERENCES

[]

REMEDIATION COMMENT

ACKNOWLEDGED: The **Aragon team** acknowledged this finding, also mentioning that:

> This plugin is designed to be a pure token voting mechanism, focused solely on governance and not on creating incentives to hold the tokens for financial purposes. It's intentionally meant to separate governance from tokenomics. Additionally, we don't have any plans in our roadmap to introduce such features in this plugin.

HAL-006 SOLVED

Unreachable condition in StagedProposalProcessor

- Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

In the StagedProposalProcessor plugin, the proposal creation reverts if the `_startDate` parameter is in the past. Later, the `proposal.lastStageTransition` set checks if the `_startDate` is zero, but that cannot happen due to the earlier assertion that `_startDate >= block.timestamp`.

Code Location

- staged-proposal-processor-plugin/src/StagedProposalProcessor.sol:

SOL 268–333

```
268 function createProposal(
269     bytes memory _metadata,
270     Action[] memory _actions,
271     uint128 _allowFailureMap,
272     uint64 _startDate,
273     bytes[][] memory _proposalParams
274 ) public virtual auth(CREATE_PROPOSAL_PERMISSION_ID) returns (uint256 proposalId) {
        // If `currentConfigIndex` is 0, this means the plugin was installed
275     // with empty configurations and still hasn't updated stages
        // in which case we should revert.
276     uint16 index = getCurrentConfigIndex();
277     if (index == 0) {
278         revert Errors.StageCountZero();
279     }
280
281     proposalId = _createProposalId(keccak256(abi.encode(_actions, _metadata)));
282
283     Proposal storage proposal = proposals[proposalId];

284     if (_proposalExists(proposal)) {
285         revert Errors.ProposalAlreadyExists(proposalId);
286     }
287
288     proposal.allowFailureMap = _allowFailureMap;
289     proposal.targetConfig = getTargetConfig();
290
291     // store stage configuration per proposal to avoid
292     // changing it while proposal is still open
293     proposal.stageConfigIndex = index;
294
295     // If the start date is in the past, revert.
296     if (_startDate < uint64(block.timestamp)) {
297         revert Errors.StartDateInvalid(_startDate);
298     }
299
300     proposal.lastStageTransition = _startDate == 0 ? uint64(block.timestamp) :
301 _startDate;
302
303     for (uint256 i = 0; i < _actions.length; ++i) {
            proposal.actions.push(_actions[i]);
304     }
305
306     // To reduce the gas costs significantly, don't store the very
307     // first stage's params in storage as they only get used in this
308     // current tx and will not be needed later on for advancing.
```

```
309     for (uint256 i = 1; i < _proposalParams.length; ++i) {
            for (uint256 j = 0; j < _proposalParams[i].length; ++j)
310             createProposalParams[proposalId][uint16(i)][j] = _proposalParams[i][j];
        }
311
312     _createBodyProposals(
313         proposalId,
314         0,
            proposal.lastStageTransition,
315         _proposalParams.length > 0 ? _proposalParams[0] : new bytes[](0)
316     );
317
318     emit ProposalCreated({
319         proposalId: proposalId,
320         creator: _msgSender(),
321         startDate: proposal.lastStageTransition,
            endDate: 0,
322         metadata: _metadata,
323         actions: _actions,
324         allowFailureMap: _allowFailureMap
325     });
326 }
327
```

RECOMMENDATION

It is recommended to review if the unreachable condition is wanted, and in that case, fix the logic to make it accessible.

REFERENCES

[]

REMEDIATION COMMENT

SOLVED: The issue was fixed in the specified commit.

HAL-007 ACKNOWLEDGED

Anyone can report a proposal result

- Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

In the StagedProposalProcessor plugin, each plugin (token voting, multisig, admin...) participating in the main proposal stage creates independent proposals that are programmed to call the StagedProposalProcessor's reportProposalResult function as a callback one they are passed and executed, so that the latter can record the status of each sub plugin result.

It was found that this function is not restricted by access control, unlike most of the plugins that priorly distribute rights for each actors. The current logic does not allow any user to modify status of real proposals but could be causing confusion in external systems working with the events.

Code Location

- staged-proposal-processor-plugin/src/StagedProposalProcessor.sol:

SOL 402-435

```
402 function reportProposalResult(
403     uint256 _proposalId,
404     uint16 _stageId,
405     ResultType _resultType,
406     bool _tryAdvance
407 ) external virtual {
408     Proposal storage proposal = proposals[_proposalId];
409
410     if (!_proposalExists(proposal)) {
411         revert Errors.NonexistentProposal(_proposalId);
412     }
413
414     uint16 currentStage = proposal.currentStage;
415
416     // Ensure that result can not be submitted
417     // for the stage that has not yet become active.
418     if (_stageId > currentStage) {
419         revert Errors.StageIdInvalid(currentStage, _stageId);
420     }
421
422     _processProposalResult(_proposalId, _stageId, _resultType);
423
424     if (_tryAdvance && _canProposalAdvance(_proposalId)) {
425         // If it's the last stage, only advance(i.e execute) if
426         // caller has permission. Note that we don't revert in
427         // this case to still allow the records being reported.
428         if (
429             proposal.currentStage != stages[proposal.stageConfigIndex].length - 1 ||
                hasExecutePermission()
430         ) {
431             _advanceProposal(_proposalId);
432         }
433     }
434 }
435
```

- staged-proposal-processor-plugin/src/StagedProposalProcessor.sol:

SOL 663-672

```sol
function _processProposalResult(
    uint256 _proposalId,
    uint16 _stageId,
    ResultType _resultType
) internal virtual {
    address sender = _msgSender();

    bodyResults[_proposalId][_stageId][sender] = _resultType;
    emit ProposalResultReported(_proposalId, _stageId, sender);
}
```

RECOMMENDATION

It is recommended to add a role to restrict proposal result reporting to only the subbodies.

REFERENCES

[]

REMEDIATION COMMENT

ACKNOWLEDGED: The **Aragon team** acknowledged this finding.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.